

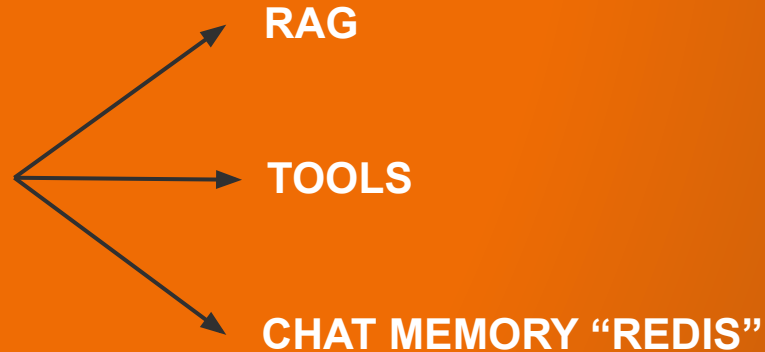
# The Agent Is a Loop

Author: Konrad Kowalczyk

Repository: [ai-agents-JAVA-SPRING](#)

# Let's start from the beginning

This is a continuation of last year's presentation, which focused on...



OLD REPO → [xkondix AI Sandbox](#)

# Roadmap

## 1. The Agent Is a Loop

*Writing one by hand, and what frameworks hide when they write it for you*

## 2. Agentic Workflow Patterns

*Five shapes, two frameworks, and when the DSL earns its keep*

## 3. MCP as a Process Boundary

*Tools, resources and prompts over two transports: HTTP for agents, STDIO for Claude Desktop*

## 4. How Do You Know It Works?

*Traces, metrics, logs and dashboards for everything your agents actually do*

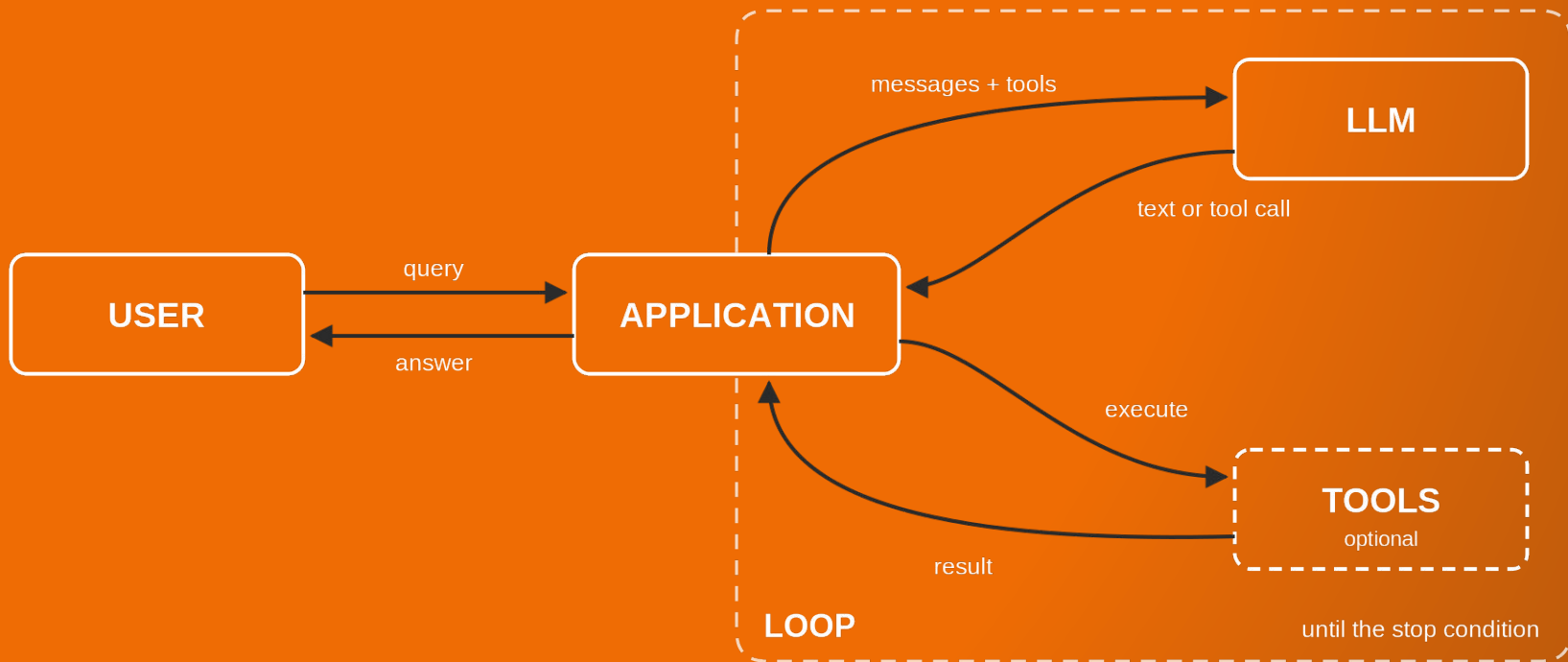
# The Agent Is a Loop

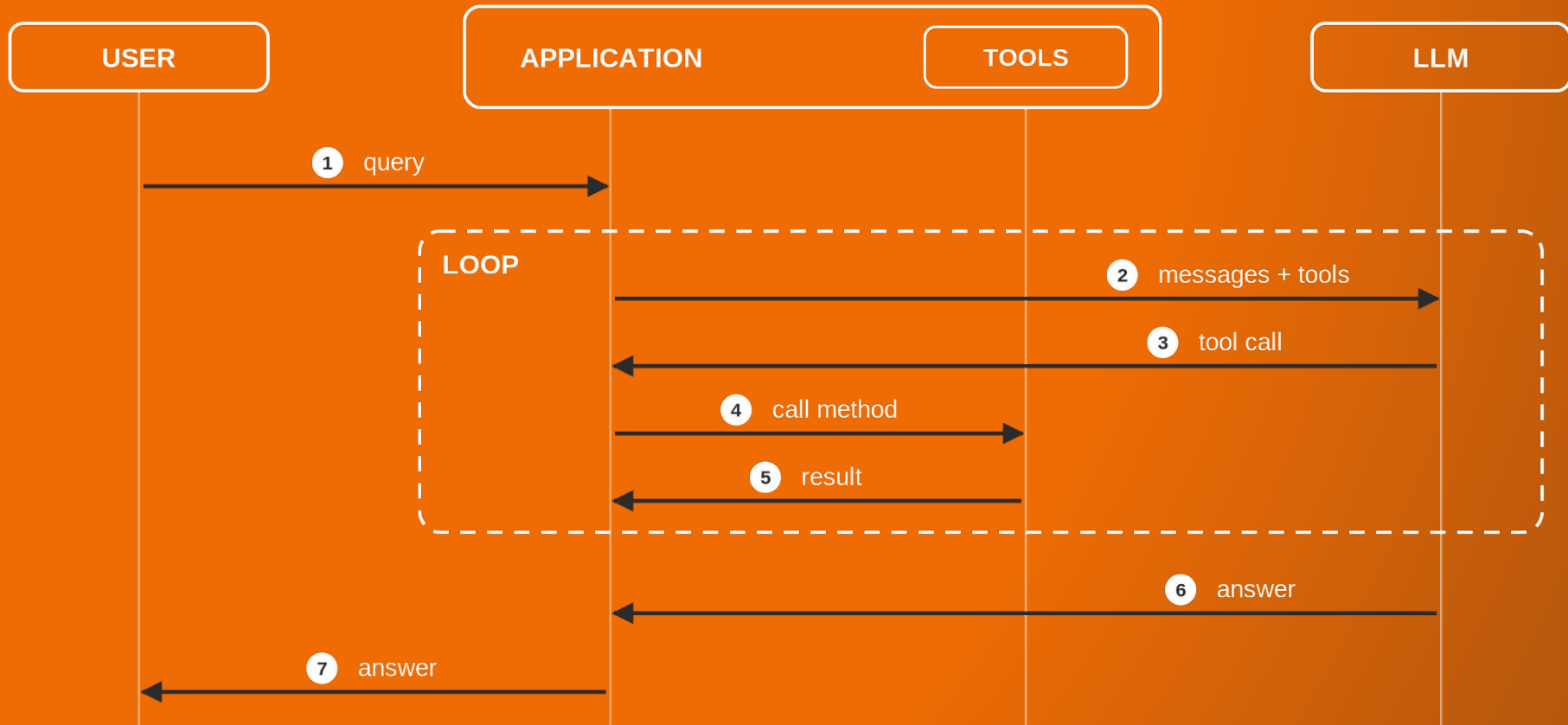
1. **What is an agent?**
2. **Build it** -> No framework
3. **Hide it** -> LangChain4j
4. **Control it** -> Spring AI
5. **Mix it** -> One provider,  $\infty$  protocols



# What is an agent?







# Raw Agent

The loop exits when the model stops requesting tools.

stop condition = no more tool calls

sources  openai  
strandsagents

```
// — Agent loop —  
for (int i = 1; i <= MAX_ITERATIONS; i++) {  
    log.info("[LOOP] Iteration {} - {} messages in history", i, history.size());  
  
    // — Step 2: Call LLM —  
    var response = llmClient.chat(history, tools.getDefinitions());  
    var message = response.firstMessage();  
  
    if (message == null) {  
        log.error("[LOOP] Empty response from LLM*");  
        return "Error: empty response from model.";  
    }  
  
    history.add(Message.assistant(message.content(), message.toolCalls()));  
  
    if (response.usage() != null) {  
        log.info("[LOOP] Tokens: input={} output={} total={}",  
            response.usage().promptTokens(),  
            response.usage().completionTokens(),  
            response.usage().totalTokens());  
    }  
  
    // — Step 3: Tool calls? —  
    if (!response.hasToolCalls()) {  
        log.info("[LOOP] Done after {} iteration(s)", i);  
        return message.content() != null ? message.content() : "";  
    }  
  
    for (ToolCall toolCall : message.toolCalls()) {  
        String result = executeToolWithSpan(toolCall, i);  
        history.add(Message.tool(toolCall.id(), result));  
    }  
}  
  
return "Max iterations (" + MAX_ITERATIONS + ") reached.";
```

# The request body structure

Every exchange with the model is JSON. The framework only decides who writes it.

- MODEL** → the model that handles this request
- MESSAGES** → the whole conversation, resent on every call
- TOOLS** → your Java methods, exposed to the model
- TEMPERATURE** → lower: more deterministic  
higher: more random
- STREAM** → false: one complete response  
true: chunks as they are generated

```
{  
  "model": "openai/gpt-4o-mini",  
  "messages": [ ... ],  
  "tools": [ ... ],  
  "temperature": 0.3,  
  "stream": false  
}
```

# The request body as JSON

## MESSAGES []

**SYSTEM** → the instructions

**USER** → the question

**ASSISTANT** → a request for a tool

**TOOL** → the result your code produced

**ASSISTANT** → the answer

LOOP

```
{
  "model": "openai/gpt-4o-mini",
  "messages": [ {
    "role": "system",
    "content": "You are a helpful assistant. Use available tools when appropriate."
  }, {
    "role": "user",
    "content": "How is the weather in Katowice?"
  }, {
    "role": "assistant",
    "tool_calls": [ {
      "id": "call_ElrK029Ec5883Ljw94tPmqcl",
      "type": "function",
      "function": {
        "name": "getWeather",
        "arguments": "{ \"city\": \"Katowice\"}"
      }
    }
  ]
}, {
  "role": "tool",
  "content": "Weather in Katowice: 22C, sunny",
  "tool_call_id": "call_ElrK029Ec5883Ljw94tPmqcl"
} ],
  "tools": [ {
    "type": "function",
    "function": {
      "name": "getCurrentTime",
      "description": "Returns the current date and time",
      "parameters": {
        "type": "object",
        "properties": { },
        "required": [ ]
      }
    }
  }, {
    "type": "function",
    "function": {
      "name": "calculateSquare",
      "description": "Calculates the square of a given number",
      "parameters": {
        "type": "object",
        "properties": {
          "number": {
            "type": "integer",
            "description": "The number to square"
          }
        },
        "required": [ "number" ]
      }
    }
  }, {
    "type": "function",
    "function": {
      "name": "getWeather",
      "description": "Gets mock weather for a city",
      "parameters": {
        "type": "object",
        "properties": {
          "city": {
            "type": "string",
            "description": "Name of the city"
          }
        },
        "required": [ "city" ]
      }
    }
  ]
},
  "temperature": 0.3,
  "stream": false
}
```

# The request body as JSON

## MESSAGES []

**SYSTEM** → written by dev, before the conversation

**USER** → written by the user

**ASSISTANT** → written by the model

**TOOL** → written by your app

**ASSISTANT** → written by the model

┌  
└

**LOOP**

# SYSTEM ROLE

**The system prompt is the job description.**

Written once by a human, resent on every iteration.

```
{  
  "role": "system",  
  "content": "You are a helpful assistant. Use available tools when appropriate."  
}
```

# USER ROLE

## The question.

Written by the user, resent on every iteration.

```
{  
  "role": "user",  
  "content": "How is the weather in Katowice?"  
}
```

# ASSISTANT ROLE

**The model either asks for a tool, or gives the final answer.**  
A tool request means the loop continues.

```
{  
  "role": "assistant",  
  "tool_calls": [{  
    "id": "call_EIrKGD9E",  
    "function": {  
      "name": "getWeather",  
      "arguments": "{\"city\":\"Katowice\"}"  
    }  
  }]  
}
```

# TOOL ROLE

**The result. Written by your app, never by the model.**

tool\_call\_id matches the request, because the model may ask for several at once.

```
{  
  "role": "tool",  
  "content": "Weather in Katowice: 22C, sunny",  
  "tool_call_id": "call_EIrKGD9E"  
}
```

# TOOLS

The functions the model may ask you to call.

Name, description and JSON schema that's all it knows about your code.

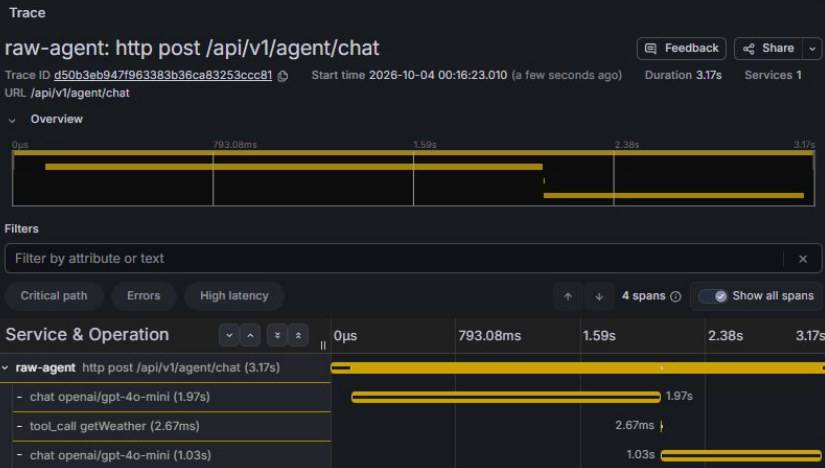
```
{
  "type": "function",
  "function": {
    "name": "getWeather",
    "description": "Gets mock weather for a city",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string",
          "description": "Name of the city"
        }
      },
      "required": ["city"]
    }
  }
}
```

```
package com.xkondix.rawagent.model;

import com.fasterxml.jackson.annotation.JsonInclude;
import tools.jackson.databind.node.ObjectNode;

19 usages  Konrad
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ToolDefinition(
    no usages
    String type,
    3 usages
    FunctionDef function
) {
    4 usages  Konrad
    public record FunctionDef(
        2 usages
        String name,
        no usages
        String description,
        1 usage
        ObjectNode parameters
    ) {}
}
```

# Observability



```
: 2026-10-04 00:16:26.131 DEBUG [LLM] Response status=200
: 2026-10-04 00:16:26.131 INFO [LOOP] Done after 2 iteration(s)
: 2026-10-04 00:16:26.131 INFO [LOOP] Tokens: input=150 output=15 total=165
: 2026-10-04 00:16:25.114 DEBUG [LLM] POST https://openrouter.ai/api/v1/chat/completions
: 2026-10-04 00:16:25.103 DEBUG [LLM] OpenAI request: model=openai/gpt-4o-mini messages=4 tools=3
: 2026-10-04 00:16:25.103 INFO [LOOP] Tool result: Weather in Katowice: 22C, sunny
: 2026-10-04 00:16:25.103 INFO [LOOP] Tool call: getWeather args={"city":"Katowice"}
: 2026-10-04 00:16:25.103 INFO [LOOP] Iteration 2 - 4 messages in history
: 2026-10-04 00:16:25.103 INFO [LOOP] Tokens: input=115 output=16 total=131
: 2026-10-04 00:16:25.103 INFO [TOOL] Executing: getWeather args={"city":"Katowice"}
: 2026-10-04 00:16:25.082 DEBUG [LLM] Response status=200
: 2026-10-04 00:16:23.148 DEBUG [LLM] POST https://openrouter.ai/api/v1/chat/completions
: 2026-10-04 00:16:23.132 DEBUG [LLM] OpenAI request: model=openai/gpt-4o-mini messages=2 tools=3
: 2026-10-04 00:16:23.132 INFO [LOOP] Iteration 1 - 2 messages in history
: 2026-10-04 00:16:23.132 INFO [LOOP] Starting. User: How is the weather in Katowice?
```

# LangChain4j Agent

Everything AiServices does, LangChain4j also lets you do yourself.

Same library, same tools, same model. Only the level of abstraction moves.



# LangChain4j Abstraction

**ChatModel** → you run the loop

**AiServices** → the framework runs it

**AgenticServices** → you declare the workflow [part II]

# LangChain4j Loop

with AiServices

```
this.assistant = AiServices.builder(Assistant.class)
    .chatModel(model)
    .toolProvider(toolProvider)
    .maxToolCallingRoundTrips(10) //by default 100
    .chatMemoryProvider(memoryId ->
        MessageWindowChatMemory.withMaxMessages(20))
    .build();
```

`.toolProvider(toolProvider)` is a custom provider,  
wrapped for observability

`.tools(demoTools)` is simpler, but  
invisible in traces

without AiServices

```
while (iteration++ < maxIterations) {
    log.info("[RAW LOOP] Iteration {} - {} messages", iteration, messages.size());

    ChatResponse response = chatModel.chat(
        ChatRequest.builder()
            .messages(messages)
            .parameters(ChatRequestParameters.builder() Builder<capture of ?>
                .toolSpecifications(toolExecutor.getToolSpecs()) capture of ?
                .build())
            .build());

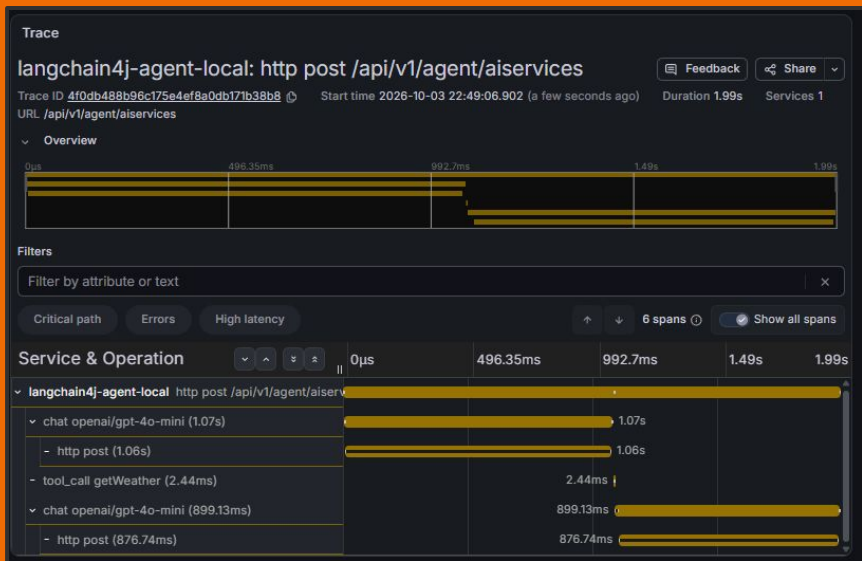
    AiMessage aiMessage = response.aiMessage();
    messages.add(aiMessage);

    if (!aiMessage.hasToolExecutionRequests()) {
        log.info("[RAW LOOP] Done after {} iterations", iteration);
        return aiMessage.text();
    }

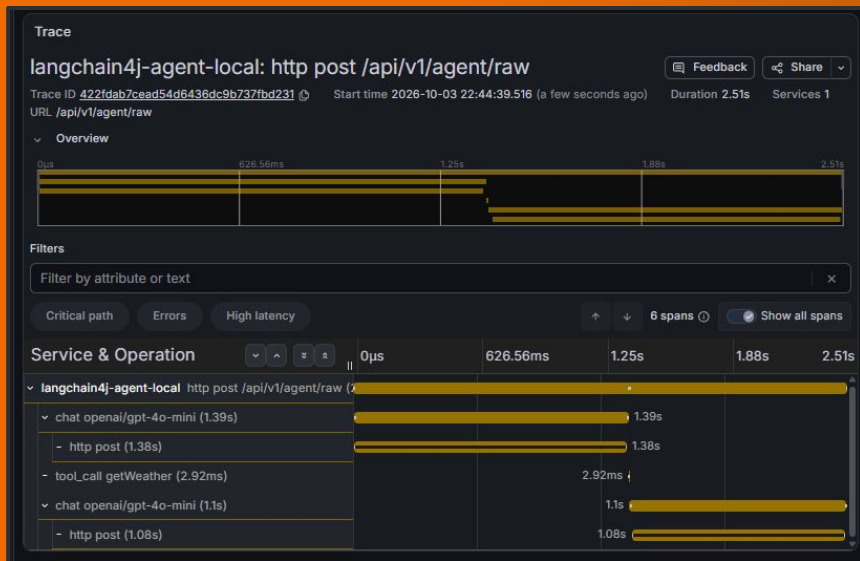
    for (var toolRequest : aiMessage.toolExecutionRequests()) {
        String result = executeToolWithSpan(
            toolRequest.name(), toolRequest.arguments(), iteration);
        messages.add(ToolExecutionResultMessage.from(toolRequest, result));
    }
}
```

# Traces

with AiServices



without AiServices

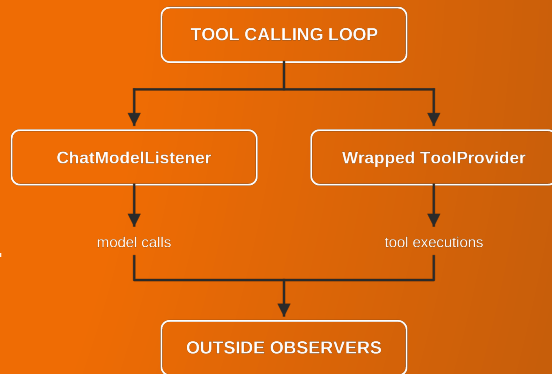


# Logs

**The loop is still observable, just from the outside.**

A `ChatModelListener` sees every model call and tool request.

A wrapped `ToolProvider` sees every tool execution.



with AiServices

```
: 2026-10-03 22:49:07.980 INFO Tool executed: getWeather(Katowice)
: 2026-10-03 22:49:06.904 INFO AiServices chat: userId=user-8gvin9
```

without AiServices

```
: 2026-10-03 22:44:42.018 INFO [RAW LOOP] Done after 2 iterations
: 2026-10-03 22:44:40.916 INFO [RAW LOOP] Iteration 2 - 4 messages
: 2026-10-03 22:44:40.914 INFO [RAW LOOP] Result: Weather in Katowice: 20C, sunny
: 2026-10-03 22:44:40.913 INFO Tool executed: getWeather(Katowice)
: 2026-10-03 22:44:40.911 INFO [RAW LOOP] Tool: getWeather args={"city":"Katowice"}
: 2026-10-03 22:44:39.520 INFO [RAW LOOP] Iteration 1 - 2 messages
```

# AgenticServices

More in chapter II

```
var translator = AgenticServices.agentBuilder(Translator.class) AgentBuilder<Translator, AgentBuilder<...>>
    .chatModel(chatModel) AgentBuilder<Translator, capture of ?>
    .maxToolCallingRoundTrips(10) capture of ?
    .outputKey("translation")
    .build();

this.workflow = AgenticServices.sequenceBuilder(ScoutReportWorkflow.class)
    .subAgents(analyst, condenser, translator)
    .outputKey(s: "translation")
    .build();
```

# Spring AI

Spring AI doesn't hide the loop, it puts it in a chain you can reach

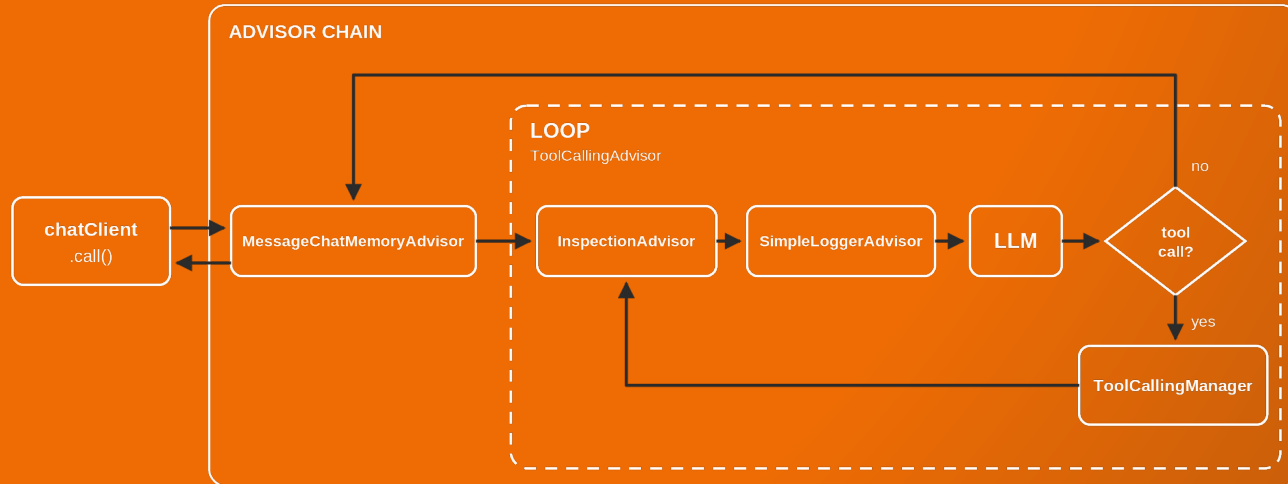
Same chain, same client, same model. Only one advisor is the loop.



# What is an Advisor?

An Advisor is middleware between your application and the AI model. It works similarly to an aspect, allowing you to add behavior around AI interactions.

It can modify requests, process responses, and control the flow of interactions with the model, keeping this logic separate from your application code.



# Advisor Code Snippet

```
public String chatWithAdvisors(String conversationId, String message) {  
    log.info("Spring AI chat [advisors]: conversationId={}", conversationId);  
  
    return chatClient.prompt() ChatClientRequestSpec  
        .user(message)  
        .advisors(  
            new InspectionAdvisor(),  
            new SimpleLoggerAdvisor(),  
            MessageChatMemoryAdvisor.builder(chatMemory).build()  
        ).advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))  
        .tools(demoFunctions)  
        .call() CallResponseSpec  
        .content();  
}
```

Adding advisors to the chain

Parameter configuration for advisors (linking an identifier with a message)

# Before and After

```
@Override
public int getOrder() {
    return 0;
}

@Override
public ChatClientRequest before(
    ChatClientRequest request,
    AdvisorChain advisorChain) {

    log.info("[Advisor] BEFORE - {} messages, user=\"{}\\\"",
        request.prompt().getInstructions().size(),
        request.prompt().getUserMessage().getText());

    return request;
}

@Override
public ChatClientResponse after(
    ChatClientResponse response,
    AdvisorChain advisorChain) {

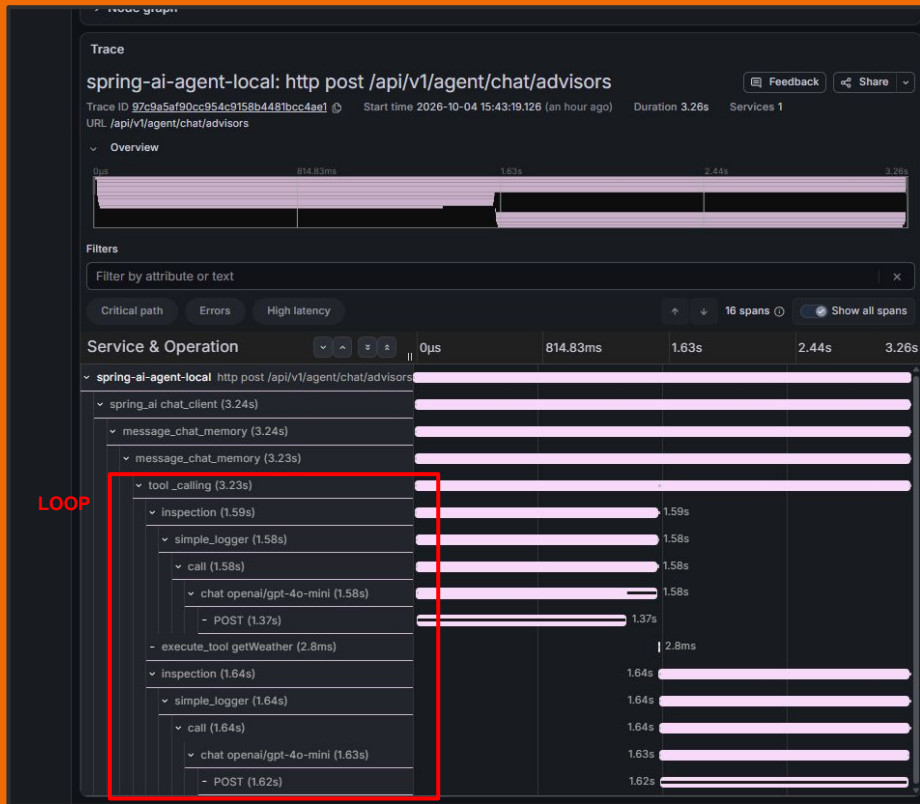
    var result = response.chatResponse().getResult();
    String text = result.getOutput().getText();
    boolean asksForTool = result.getOutput().hasToolCalls();

    log.info("[Advisor] AFTER - toolCalls={}, responseLength={},
        asksForTool, text != null ? text.length() : 0);

    return response;
}
```

```
ected_level scope_name Show original line
: 2026-10-04 18:46:14.824 INFO info com.xkondix.springai.agent.advisor.InspectionAdvisor
: 2026-10-04 18:46:14.822 DEBUG debug org.springframework.ai.chat.client.advisor.SimpleLoggerAdvisor
: 2026-10-04 18:46:14.820 INFO info org.springframework.ai.chat.observation.ChatModelCompletionObservat
ionHandler
: 2026-10-04 18:46:14.820 INFO info org.springframework.ai.chat.observation.ChatModelPromptContentObser
vationHandler
: 2026-10-04 18:46:13.053 DEBUG debug org.springframework.ai.chat.client.advisor.SimpleLoggerAdvisor
: 2026-10-04 18:46:13.050 INFO info com.xkondix.springai.agent.advisor.InspectionAdvisor
: 2026-10-04 18:46:13.046 INFO info com.xkondix.springai.agent.tools.DemoFunctions
: 2026-10-04 18:46:13.036 INFO info com.xkondix.springai.agent.advisor.InspectionAdvisor 1
: 2026-10-04 18:46:13.035 DEBUG debug org.springframework.ai.chat.client.advisor.SimpleLoggerAdvisor 2
: 2026-10-04 18:46:12.983 INFO info org.springframework.ai.chat.observation.ChatModelCompletionObservat
ionHandler
: 2026-10-04 18:46:12.983 INFO info org.springframework.ai.chat.observation.ChatModelPromptContentObser
vationHandler
: 2026-10-04 18:46:10.480 DEBUG debug org.springframework.ai.chat.client.advisor.SimpleLoggerAdvisor 2
: 2026-10-04 18:46:10.477 INFO info com.xkondix.springai.agent.advisor.InspectionAdvisor 1
: 2026-10-04 18:46:10.343 INFO info com.xkondix.springai.agent.service.SpringAiAgentService
```

# Advisors Inside the Loop



```
size=1.55.0 trace_id=97c9a5af90cc954c9158b4481bcc4ae1
- : 2026-10-04 15:43:22.369 INFO [Advisor] AFTER - toolCalls=false, responseLength=52
: 2026-10-04 15:43:22.369 INFO Chat Model Completion:
["The weather in Katowice is currently 20°C and sunny."]
: 2026-10-04 15:43:22.369 INFO Chat Model Prompt Content:
["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?", "", ""]
: 2026-10-04 15:43:20.734 INFO [Advisor] BEFORE - 4 messages, user="How is the weather in Katowice?"
: 2026-10-04 15:43:20.732 INFO Spring AI Tool: getWeather(Katowice)
: 2026-10-04 15:43:20.728 INFO [Advisor] AFTER - toolCalls=true, responseLength=0
: 2026-10-04 15:43:20.718 INFO Chat Model Completion:
[]
: 2026-10-04 15:43:20.718 INFO Chat Model Prompt Content:
["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?"]
: 2026-10-04 15:43:19.144 INFO [Advisor] BEFORE - 2 messages, user="How is the weather in Katowice?"
: 2026-10-04 15:43:19.129 INFO Spring AI chat [advisors]: conversationId=user-f91vbd
```

# Advisor Order

The execution order of advisors in the chain is determined by the `getOrder()` method.

Lower order = earlier in the chain,  
farther from the model [LLM].

Higher order = later in the chain,  
closer to the model [LLM].



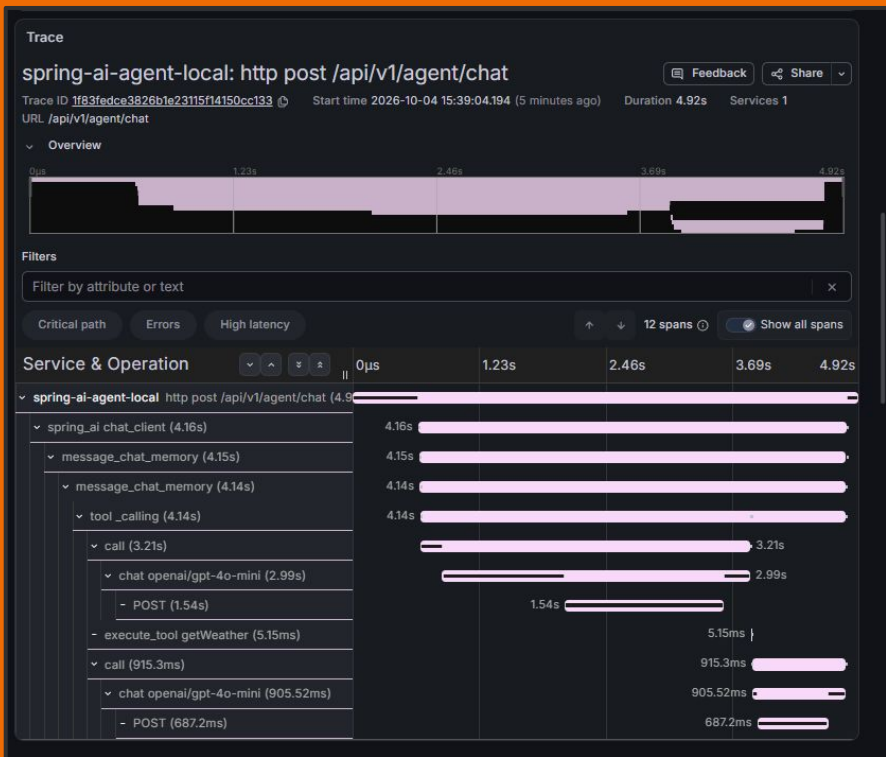
# Example of Advisors

|                                                                          | Order ID Value         |
|--------------------------------------------------------------------------|------------------------|
| <b>MessageChatMemoryAdvisor</b> → adds history as separate messages      | <b>MIN_VALUE + 200</b> |
| <b>QuestionAnswerAdvisor</b> → adds context from a vector store (RAG)    | <b>0</b>               |
| <b>SafeGuardAdvisor</b> → checks the prompt and history for banned words | <b>0</b>               |
| <b>ToolCallingAdvisor</b> → owns the tool calling lifecycle              | <b>MIN_VALUE + 300</b> |
| <b>SimpleLoggerAdvisor</b> → logs request and response information       | <b>0</b>               |

Integer.MIN\_VALUE = -2 147 483 648 ← HIGHEST\_PRECEDENCE (doc name)

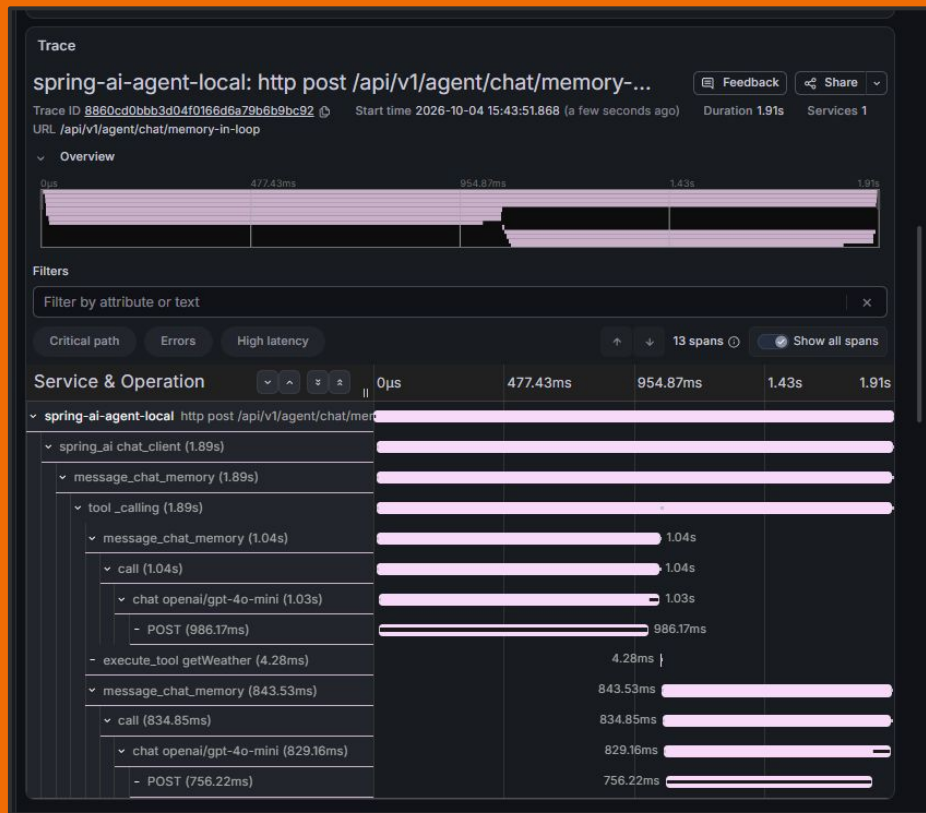
**Order 0 is not first. It is inside the loop!!!**

# Default



```
3e3826b1e23115f1
-| : 2026-10-04 15:39:08.988 INFO Chat Model Prompt Content:
    ["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?", """]
: 2026-10-04 15:39:08.987 INFO Chat Model Completion:
    ["The weather in Katowice is currently 20°C and sunny."]
: 2026-10-04 15:39:08.070 INFO Spring AI Tool: getWeather(Katowice)
: 2026-10-04 15:39:08.050 INFO Chat Model Prompt Content:
    ["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?"]
: 2026-10-04 15:39:08.049 INFO Chat Model Completion:
    []
: 2026-10-04 15:39:04.499 INFO Spring AI chat [default]: conversationId=user-bogff1
```

# Memory in the Loop



Now one of them is inside the loop.  
The docs advise against two memory  
advisors. This is why the question is  
sent twice.

```
d8bbb3d84f8166da...
+1 : 2026-10-04 15:43:53.762 INFO Chat Model Completion:
    ["The weather in Katowice is currently 20°C and sunny."]
: 2026-10-04 15:43:53.762 INFO Chat Model Prompt Content:
    ["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?", "How
    is the weather in Katowice?", "", ""]
: 2026-10-04 15:43:52.920 INFO Spring AI Tool: getWeather(Katowice)
: 2026-10-04 15:43:52.916 INFO Chat Model Prompt Content:
    ["You are a helpful assistant. Use available tools when needed.", "How is the weather in Katowice?"]
: 2026-10-04 15:43:52.916 INFO Chat Model Completion:
    []
: 2026-10-04 15:43:51.870 INFO Spring AI chat [memory in loop]: conversationId=user-gco646
```

# Three levels of hiding the loop

## BY HAND

raw-agent

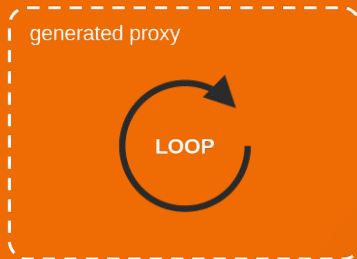
```
while (i++ < MAX) {  
    resp = model.call(messages);  
    if (!resp.hasToolCalls()) return resp;  
    for (call : resp.toolCalls())  
        messages.add(execute(call));  
}
```



## FRAMEWORK SERVICE

LangChain4j AIServices

```
interface Assistant {  
    String chat(String msg);  
}  
AIServices.builder(Assistant.class)  
    .chatModel(model).tools(tools)  
    .build();
```



## ADVISOR CHAIN

Spring AI ChatClient

```
chatClient.prompt(msg)  
    .tools(tools)  
    .call()  
    .content();
```



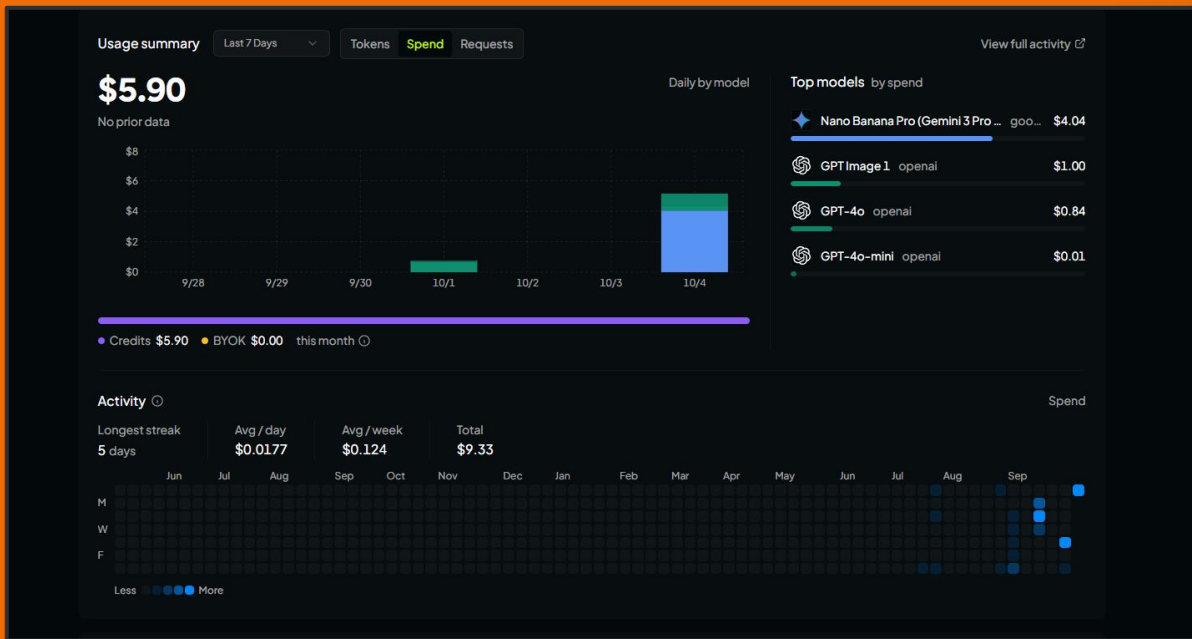
# Who stops the loop?

| Implementation  | Limit                                    | When exceeded                                          |
|-----------------|------------------------------------------|--------------------------------------------------------|
| raw-agent       | 10 iterations                            | a string in HTTP 200, indistinguishable from an answer |
| LC4j AIServices | 100 by default (10 in the demo)          | exception                                              |
| Spring AI 2.0.1 | 40 per tool / 150 per request <u>doc</u> | response with a dedicated finish reason                |

# OpenRouter

OpenRouter is a single API in front of hundreds of models from dozens of vendors.

One key, one base URL, and the OpenAI protocol everyone already speaks.



# Configuration

```
spring:
  config:
    activate:
      on-profile: openrouter
  ai:
    model:
      chat: openai
    openai:
      base-url: https://openrouter.ai/api/v1
      api-key: ${OPENROUTER_API_KEY}
      chat:
        options:
          model: ${OPENROUTER_MODEL:openai/gpt-4o-mini}
          temperature: 0.3
      image:
        options:
          model: ${OPENROUTER_IMAGE_MODEL:google/gemini-3-pro-image}
          size: 1024x1024
      audio:
        speech:
          options:
            model: ${OPENROUTER_TTS_MODEL:x-ai/grok-voice-tts-1.0}
            voice: ${OPENROUTER_TTS_VOICE:eve}
            response-format: mp3
            speed: 1.0
        transcription:
          options:
            model: ${OPENROUTER_STT_MODEL:openai/whisper-large-v3}
```

Spring AI configuration



Own properties



```
multimodal:
  media-dir: ./media
  vision-model: ${OPENROUTER_VISION_MODEL:openai/gpt-4o}
  music:
    model-pro: ${OPENROUTER_MUSIC_MODEL_PRO:google/lyria-3-pro-preview}
    model-clip: ${OPENROUTER_MUSIC_MODEL_CLIP:google/lyria-3-clip-preview}
  video:
    enabled: ${MULTIMODAL_VIDEO_ENABLED:false}
    model: ${OPENROUTER_VIDEO_MODEL:google/veo-3.1}
    resolution: ${OPENROUTER_VIDEO_RESOLUTION:720p}
```

# Chat

```
public String chat(String conversationId, String question, String imageUrl) {
    String message = (imageUrl == null || imageUrl.isBlank())
        ? question
        : question + "\n\n[The user attached an image at " + imageUrl + "];

    log.info("[MM] router conversationId={} image={} question='{}'",
        conversationId, imageUrl, question);

    List<Object> toolBeans = new ArrayList<>(List.of(tools, musicTools));
    if (videoTools != null) {
        toolBeans.add(videoTools);
    }

    return router.prompt() ChatClientRequestSpec
        .advisors(MessageChatMemoryAdvisor.builder(chatMemory).build())
        .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, conversationId))
        .tools(toolBeans.toArray())
        .user(message)
        .call() CallResponseSpec
        .content();
}
```

# Image Generation

```
try {
    ImageResponse response = imageModel.call(new ImagePrompt(description));
    var image = response.getResult().getOutput();

    byte[] bytes;
    if (image.getB64Json() != null && !image.getB64Json().isBlank()) {
        bytes = Base64.getDecoder().decode(image.getB64Json());
    } else if (image.getUrl() != null && !image.getUrl().isBlank()) {
        bytes = download(image.getUrl());
    } else {
        collector.recordFailure();
        record(modality: "image", bytes: 0, sample);
        return "ERROR: the image model returned neither b64_json nor a url. Do NOT retry.";
    }
}
```

# Speech

```
public String speak_text(  
    @ToolParam(description = "The exact text to read out loud, in the language it is written in")  
    String text) {  
  
    String refusal = refuseIfBudgetSpent(tool: "speak_text", capability: "read text out loud");  
    if (refusal != null) {  
        return refusal;  
    }  
  
    log.info("[MM] speak_text: {} chars", text == null ? 0 : text.length());  
    Timer.Sample sample = Timer.start(registry);  
    try {  
        byte[] mp3 = speechModel.call(text);  
        String url = storage.store(mp3, extension: "mp3");  
        String scriptUrl = storage.store(text.getBytes(StandardCharsets.UTF_8), extension: "txt");  
        collector.add(Artifact.of(scriptUrl, Artifact.Kind.TEXT, abbreviate(text)));  
        collector.add(Artifact.of(url, Artifact.Kind.SPEECH, abbreviate(text)));  
  
        record(modality: "speech", mp3.length, sample);  
        return "Done — the audio has been generated and is playing for the user.";  
    } catch (RuntimeException e) {  
        int failures = collector.recordFailure();  
        log.error("[MM] speak_text failed ({} / {}): {}",  
            failures, MediaCollector.MAX_FAILURES, e.getMessage());  
        record(modality: "speech", bytes: 0, sample);  
        return "ERROR: could not synthesize speech — " + e.getMessage()  
            + ". Do NOT retry; tell the user this part failed.";  
    }  
}
```

# Vision

```
public String analyze(String imageUrl, String question) {
    String relative = imageUrl.replace(MediaStorage.URL_PREFIX, replacement: "");
    Path file = storage.resolve(relative);
    log.info("[MM] analyze_image {} question='{}'", relative, question);

    Timer.Sample sample = Timer.start(registry);
    try {
        MimeType mime = relative.endsWith(".png")
            ? MimeTypeUtils.IMAGE_PNG : MimeTypeUtils.IMAGE_JPEG;

        String answer = vision.prompt() ChatClientRequestSpec
            .options(OpenAiChatOptions.builder().model(model))
            .user(u -> u
                .text(question == null || question.isBlank()
                    ? "Describe this image."
                    : question)
                .media(new Media(mime, new FileSystemResource(file))))
            .call() CallResponseSpec
            .content();

        record(sample);
        return answer;
    } catch (RuntimeException e) {
        log.error("[MM] analyze_image failed: {}", e.getMessage());
        record(sample);
        throw e;
    }
}
```

# Music

Here is where the trouble started....

**Lyria is a chat endpoint. Getting the bytes out took five attempts.**

1. `chatModel.call()` → timed lyrics, no audio “[0.0:7.3] O-Zone, we dance tonight...”
2. `+ getMedia()` → media blocks = 0
3. `+ modalities` → `voice` is required, but was not set
4. `+ voice` → 400: Audio output requires stream: true
5. `+ stream()` → the stream arrives, media blocks = 0

Every one of those errors was correct and specific. None of them said where to look.

# Why can't Spring AI reach it?

| Layer      | Behavior                                                      |
|------------|---------------------------------------------------------------|
| Google     | Exposes Lyria through the Interactions API, not /audio/speech |
| OpenRouter | Returns audio from chat only when stream: true                |
| OpenAI SDK | Validates a speech-oriented request schema and requires voice |
| Spring AI  | Maps audio to Media only in the non-streaming (blocking) path |

**No layer made a mistake, so no layer could warn.**

Music dropped to raw HTTP. One modality on the transport is cheaper than pretending the abstraction covers five.

# Code Snippet

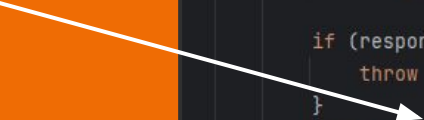
```
public Result generate(String model, String prompt) {
    Map<String, Object> body = new LinkedHashMap<>();
    body.put("model", model);
    body.put("messages", List.of(Map.of(k1: "role", v1: "user", k2: "content", prompt)));
    body.put("modalities", List.of("text", "audio"));
    body.put("audio", Map.of(k1: "format", v1: "mp3"));
    body.put("stream", true);

    HttpRequest request = HttpRequest.newBuilder(URI.create(baseUrl + "/chat/completions"))
        .header(name: "Authorization", value: "Bearer " + apiKey)
        .header(name: "Content-Type", value: "application/json")
        .header(name: "Accept", value: "text/event-stream")
        .timeout(Duration.ofMinutes(5))
        .POST(HttpRequest.BodyPublishers.ofString(mapper.writeValueAsString(body)))
        .build();

    try {
        HttpResponse<Stream<String>> response =
            http.send(request, HttpResponse.BodyHandlers.ofLines());

        if (response.statusCode() != 200) {
            throw new IllegalStateException("Lylia returned HTTP " + response.statusCode());
        }
        return consume(response.body());
    }
}
```

Bytes

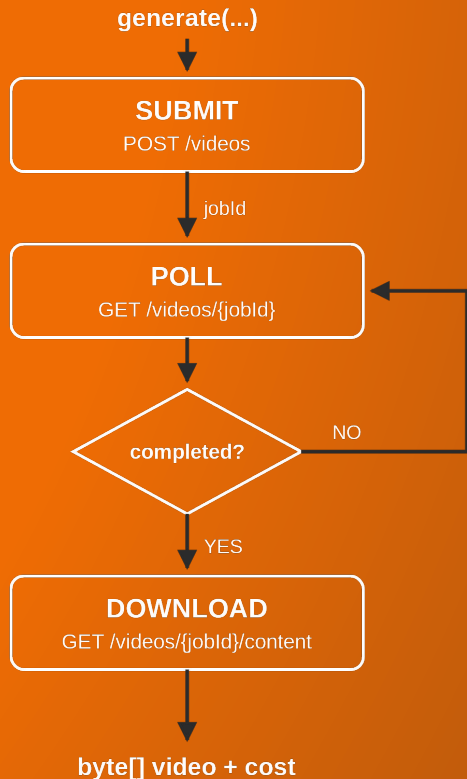


# Video

**Video generation is asynchronous: submit, poll, download.**

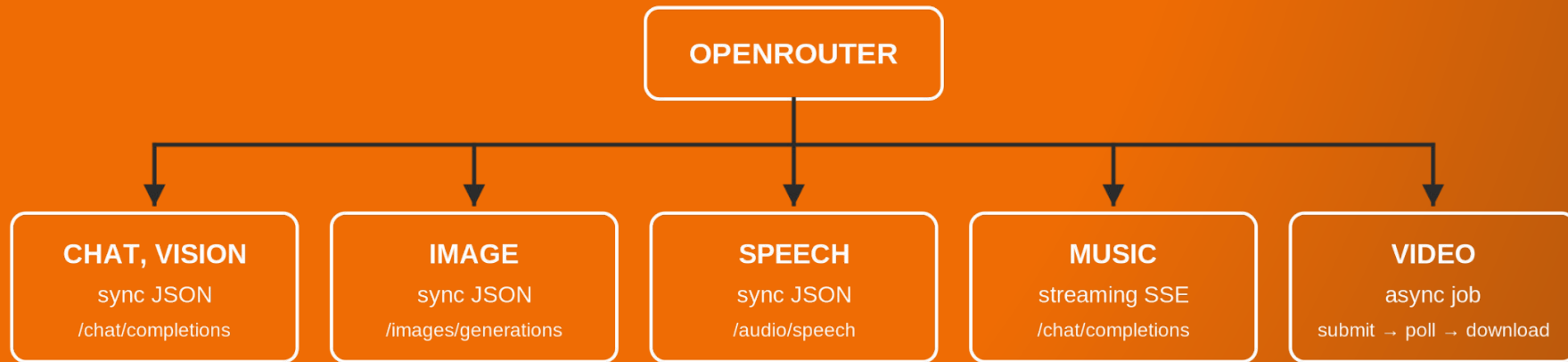
There is no `VideoModel` in Spring AI, so the client is custom.

Music had the opposite problem, the abstraction existed and could not be reached.



# Is one protocol really enough?

One provider does not mean one protocol.



See you in part II next week!

